Claude Code Docs

配置 › 管理 Claude 的内存

配置

管理 Claude 的内存

复制页面

了解如何通过不同的内存位置和最佳实践来管理 Claude Code 跨会话的内存。

Claude Code 有两种在会话间持久化的内存：

自动内存：Claude 自动保存有用的上下文，如项目模式、关键命令和您的偏好设置。这在会话间持久化。

CLAUDE.md 文件：您编写和维护的 Markdown 文件，包含 Claude 应遵循的说明、规则和偏好设置。

两者都在每个会话开始时加载到 Claude 的上下文中，尽管自动内存仅加载其主文件的前 200 行。

确定内存类型

Claude Code 提供了多个分层结构的内存位置，每个位置都有不同的用途：

内存类型	位置	用途	用例示例	共享对象
托管策略	• macOS: `/Library/Application Support/ClaudeCode/CLAUDE.md` • Linux: `/etc/claude-code/CLAUDE.md` • Windows: `C:\Program Files\ClaudeCode\CLAUDE.md`	由 IT/DevOps 管理的组织范围内的说明	公司编码标准、安全策略、合规要求	组织中的所有用
项目内存	`./CLAUDE.md` 或 `./.claude/CLAUDE.`	项目的团队共享说明	项目架构、编码标准、常见工作流	通过源代码控制成员

内存类型	位置	用途	用例示例	共享对象
	`md`			
项目规则	`./.claude/rules/*.md`	模块化的、特定主题的项目说明	特定于语言的指南、测试约定、API 标准	通过源代码控制成员
用户内存	`~/.claude/CLAUDE.md`	所有项目的个人偏好设置	代码样式偏好、个人工具快捷方式	仅您（所有项目
项目内存（本地）	`./CLAUDE.local.md`	个人的项目特定偏好设置	您的 sandbox URL、首选测试数据	仅您（当前项目
自动内存	`~/.claude/projects/<project>/memory/`	Claude 的自动笔记和学习	项目模式、调试见解、架构笔记	仅您（每个项目

工作目录上方目录层次结构中的 CLAUDE.md 文件在启动时完整加载。子目录中的 CLAUDE.md 文件在 Claude 读取这些目录中的文件时按需加载。自动内存仅加载 `MEMORY.md` 的前 200 行。更具体的说明优先于更广泛的说明。

> CLAUDE.local.md 文件会自动添加到 .gitignore，使其成为不应检入版本控制的私有项目特定偏好设置的理想选择。

自动内存

自动内存是一个持久化目录，Claude 在工作时记录学习、模式和见解。与包含您为 Claude 编写的说明的 CLAUDE.md 文件不同，自动内存包含 Claude 根据会话期间的发现为自己编写的笔记。

> 自动内存默认启用。要切换它的开启或关闭，请使用 `/memory` 并选择自动内存切换。

Claude 记住的内容

当 Claude 工作时，它可能会保存以下内容：

- 项目模式：构建命令、测试约定、代码样式偏好
- 调试见解：棘手问题的解决方案、常见错误原因
- 架构笔记：关键文件、模块关系、重要抽象
- 您的偏好：沟通风格、工作流习惯、工具选择

Claude Code Docs

自动内存的存储位置

每个项目都有自己的内存目录，位于 `~/.claude/projects/<project>/memory/` 。 `<project>` 路径来自 git 存储库根目录，因此同一存储库中的所有子目录共享一个自动内存目录。Git worktrees 获得单独的内存目录。在 git 存储库外，使用工作目录代替。

该目录包含一个 `MEMORY.md` 入口点和可选的主题文件：

```
~/.claude/projects/<project>/memory/
├── MEMORY.md          # 简洁索引，加载到每个会话中
├── debugging.md       # 调试模式的详细笔记
├── api-conventions.md # API 设计决策
└── ...                # Claude 创建的任何其他主题文件
```

`MEMORY.md` 充当内存目录的索引。Claude 在整个会话中读取和写入此目录中的文件，使用 `MEMORY.md` 来跟踪存储的内容。

工作原理

`MEMORY.md` 的前 200 行在每个会话开始时加载到 Claude 的系统提示中。超过 200 行的内容不会自动加载，Claude 被指示通过将详细笔记移到单独的主题文件中来保持简洁。

主题文件如 `debugging.md` 或 `patterns.md` 在启动时不加载。Claude 在需要信息时使用其标准文件工具按需读取它们。

Claude 在会话期间读取和写入内存文件，因此您会看到内存更新在您工作时发生。

管理自动内存

自动内存文件是您可以随时编辑的 markdown 文件。使用 `/memory` 打开文件选择器，其中包括您的自动内存入口点以及您的 CLAUDE.md 文件。 `/memory` 选择器还包括一个自动内存切换，用于打开或关闭该功能。

要要求 Claude 保存特定内容，请直接告诉它：“记住我们使用 pnpm，而不是 npm”或”保存到内存中 API 测试需要本地 Redis 实例”。

您也可以通过设置或环境变量控制自动内存。

通过将 `autoMemoryEnabled` 添加到您的用户设置来禁用所有项目的自动内存：

Claude Code Docs

```
// .claude/settings.json
{ "autoMemoryEnabled": false }
```

通过将 `autoMemoryEnabled` 添加到项目设置来禁用单个项目的自动内存：

```
// .claude/settings.json
{ "autoMemoryEnabled": false }
```

使用 `CLAUDE_CODE_DISABLE_AUTO_MEMORY` 环境变量覆盖所有其他设置。这优先于 `/memory` 切换和 `settings.json` ，使其对 CI 或托管环境很有用：

```
export CLAUDE_CODE_DISABLE_AUTO_MEMORY=1  # 强制关闭
export CLAUDE_CODE_DISABLE_AUTO_MEMORY=0  # 强制打开
```

CLAUDE.md 导入

CLAUDE.md 文件可以使用 `@path/to/import` 语法导入其他文件。以下示例导入 3 个文件：

```
See @README for project overview and @package.json for avai

# Additional Instructions
- git workflow @docs/git-instructions.md
```

允许相对路径和绝对路径。相对路径相对于包含导入的文件解析，而不是工作目录。对于不应检入版本控制的私有每项目偏好设置，首选 `CLAUDE.local.md` ：它会自动加载并添加到 `.gitignore` 。

如果您在多个 git worktrees 中工作， `CLAUDE.local.md` 仅存在于其中一个。改用主目录导入，以便所有 worktrees 共享相同的个人说明：

```
# Individual Preferences
- @~/.claude/my-project-instructions.md
```

Claude Code Docs

Claude Code 第一次在项目中遇到外部导入时，会显示一个批准对话框，列出特定文件。批准以加载它们；拒绝以跳过它们。这是每个项目的一次性决定：一旦拒绝，对话框不会再次出现，导入保持禁用。

为了避免潜在的冲突，导入不会在 markdown 代码跨度和代码块内进行评估。

```
This code span will not be treated as an import: `@anthropi
```

导入的文件可以递归导入其他文件，最大深度为 5 跳。您可以通过运行 `/memory` 命令查看加载了哪些内存文件。

Claude 如何查找内存

Claude Code 递归读取内存：从 cwd 开始，Claude Code 向上递归到（但不包括）根目录 / 并读取它找到的任何 CLAUDE.md 或 CLAUDE.local.md 文件。这在大型存储库中工作时特别方便，您在 *foo/bar/* 中运行 Claude Code，并在 *foo/CLAUDE.md* 和 *foo/bar/CLAUDE.md* 中都有内存。

Claude 还会发现当前工作目录下的子树中嵌套的 CLAUDE.md。它们不是在启动时加载，而是仅在 Claude 读取这些子树中的文件时包含。

从其他目录加载内存

`--add-dir` 标志使 Claude 可以访问主工作目录外的其他目录。默认情况下，不加载这些目录中的 CLAUDE.md 文件。

要也从其他目录加载内存文件（CLAUDE.md、.claude/CLAUDE.md 和 .claude/rules/*.md），请设置 `CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD` 环境变量：

```
CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1 claude --add
```

使用 `/memory` 直接编辑内存

在会话期间使用 `/memory` 命令在系统编辑器中打开任何内存文件，以进行更广泛的添加或组织。

Claude Code Docs

›

设置项目内存

假设您想设置一个 CLAUDE.md 文件来存储重要的项目信息、约定和常用命令。项目内存可以存储在 `./CLAUDE.md` 或 `./.claude/CLAUDE.md` 中。

使用以下命令为您的代码库引导 CLAUDE.md：

```
> /init
```

提示：

包含常用命令（构建、测试、lint）以避免重复搜索

记录代码样式偏好和命名约定

添加特定于您的项目的重要架构模式

CLAUDE.md 内存可用于与您的团队共享的说明和您的个人偏好。

使用 `.claude/rules/` 的模块化规则

对于较大的项目，您可以使用 `.claude/rules/` 目录将说明组织到多个文件中。这允许团队维护专注的、组织良好的规则文件，而不是一个大的 CLAUDE.md。

基本结构

在您的项目的 `.claude/rules/` 目录中放置 markdown 文件：

```
your-project/
├── .claude/
│   ├── CLAUDE.md           # 主项目说明
│   └── rules/
│       ├── code-style.md   # 代码样式指南
│       ├── testing.md      # 测试约定
│       └── security.md     # 安全要求
```

`.claude/rules/` 中的所有 `.md` 文件都会自动加载为项目内存，优先级与 `.claude/CLAUDE.md` 相同。

Claude Code Docs

›

路径特定规则

规则可以使用带有 `paths` 字段的 YAML frontmatter 限定到特定文件。这些条件规则仅在 Claude 处理与指定模式匹配的文件时适用。

```
---
paths:
  - "src/api/**/*.ts"
---

# API Development Rules

- All API endpoints must include input validation
- Use the standard error response format
- Include OpenAPI documentation comments
```

没有 `paths` 字段的规则无条件加载，适用于所有文件。

Glob 模式

`paths` 字段支持标准 glob 模式：

模式	匹配
`**/*.ts`	任何目录中的所有 TypeScript 文件
`src/**/*`	`src/` 目录下的所有文件
`*.md`	项目根目录中的 Markdown 文件
`src/components/*.tsx`	特定目录中的 React 组件

您可以指定多个模式：

Claude Code Docs

```
paths:
  - "src/**/*.ts"
  - "lib/**/*.ts"
  - "tests/**/*.test.ts"
---
```

支持大括号扩展以匹配多个扩展名或目录：

```
---
paths:
  - "src/**/*.{ts,tsx}"
  - "{src,lib}/**/*.ts"
---

# TypeScript/React Rules
```

这会展开 `src/**/*.{ts,tsx}` 以匹配 `.ts` 和 `.tsx` 文件。

子目录

规则可以组织到子目录中以获得更好的结构：

```
.claude/rules/
├── frontend/
│   ├── react.md
│   └── styles.md
├── backend/
│   ├── api.md
│   └── database.md
└── general.md
```

所有 `.md` 文件都会被递归发现。

符号链接

`.claude/rules/` 目录支持符号链接，允许您在多个项目中共享常见规则：

Claude Code Docs

```
# 符号链接共享规则目录
ln -s ~/shared-claude-rules .claude/rules/shared

# 符号链接单个规则文件
ln -s ~/company-standards/security.md .claude/rules/security.md
```

符号链接会被解析，其内容会正常加载。循环符号链接会被检测并妥善处理。

用户级规则

您可以在 `~/.claude/rules/` 中创建适用于所有项目的个人规则：

```
~/.claude/rules/
├── preferences.md    # 您的个人编码偏好
└── workflows.md      # 您的首选工作流
```

用户级规则在项目规则之前加载，使项目规则具有更高的优先级。

`.claude/rules/` 的最佳实践：

- **保持规则专注**：每个文件应涵盖一个主题（例如 `testing.md` 、 `api-design.md` ）
- **使用描述性文件名**：文件名应指示规则涵盖的内容
- **谨慎使用条件规则**：仅在规则确实适用于特定文件类型时添加 `paths` frontmatter
- **使用子目录组织**：分组相关规则（例如 `frontend/` 、 `backend/` ）

组织级内存管理

组织可以部署适用于所有用户的集中管理的 CLAUDE.md 文件。

要设置组织级内存管理：

1. 在上面 中显示的**托管策略**位置创建托管内存文件。
2. 通过您的配置管理系统（MDM、Group Policy、Ansible 等）部署，以确保在所有开发人员机器上一致分发。

内存最佳实践

- **具体明确**：“使用 2 空格缩进”比”正确格式化代码”更好。
- **使用结构来组织**：将每个单独的内存格式化为项目符号，并在描述性 markdown 标题下分组相关内存。
- **定期审查**：随着项目的发展更新内存，以确保 Claude 始终使用最新的信息和上下文。

此页面对您有帮助吗？ 是 否

使用快速模式加快响应速度 自定义你的状态行

Claude Code Docs

Company

Anthropic
Careers
Economic Futures
Research
News
Trust center
Transparency

Help and security

Availability
Status
Support center

Learn

Courses
MCP connectors
Customer stories
Engineering blog
Events
Powered by Claude

Terms and policies

Privacy policy
Disclosure policy
Usage policy
Commercial terms
Consumer terms

Service partners

Claude Code Docs

Startups program